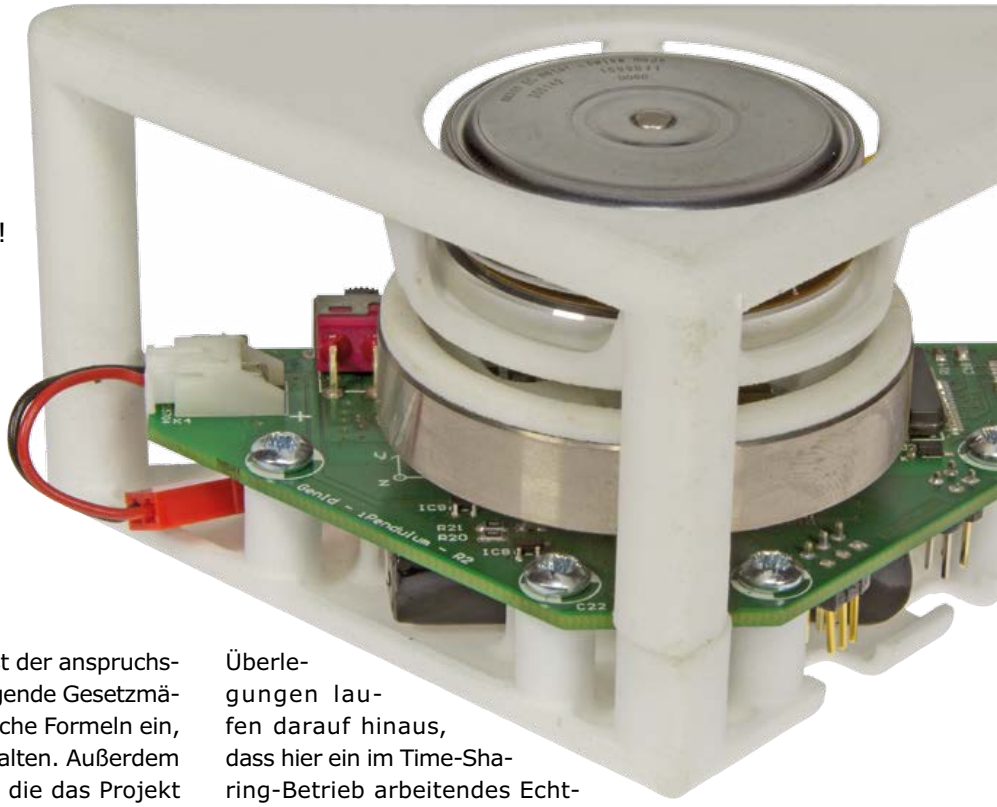
i-Pendel

Teil 2: Software, Aufbau und Diagnose

Von **Jean-Sébastien Gonsette** (Belgien)

Beim letzten Mal haben wir den theoretischen Hintergrund dieses Gleichgewichtskünstlers beleuchtet, jetzt ist die Zeit der Praxis gekommen! Wir weihen Sie in die Finessen der regelungs-technischen und mechanischen Konstruktion ein. Nach Vollendung des Werks können Sie die inneren Abläufe des i-Pendels mit einem dafür entwickelten Diagnose-Programm ergründen.



Software

Die Embedded-Software des Mikrocontrollers ist der anspruchsvollste Teil dieses Projekts. Dort fließen grundlegende Gesetzmäßigkeiten, Simulationsmodelle und mathematische Formeln ein, die zusammen das i-Pendel im Gleichgewicht halten. Außerdem sind noch Hintergrundaktivitäten nötig, ohne die das Projekt nicht laufen würde. Es geht um die Kommunikation mit den Sensoren, die Motorsteuerung und die Kommunikation mit dem PC, so dass die Telemetrie-Daten (Sensor-Signale, Drehmomente, i-Pendel-Positionen, Akku-Daten...) dort betrachtet und analysiert werden können. Diese Tasks dürfen die Regelung nicht ausbremsen. Die AD-Wandlung der Sensor-Signale dauert ungefähr 10 ms. Adäquates Reagieren auf Sensor-Signale erfordert zwar wenig Rechenleistung, doch die Firmware muss darauf möglichst schnell reagieren. Dies bedeutet, dass Nebenfunktionen wie die Kommunikation mit dem UART nicht in die Regelungsschleife einbezogen werden dürfen. Die Routinen dieser Funktionen werden ständig unterbrochen, sie müssen ihre Aufgaben trotzdem erfüllen.

Das Schreiben eines solchen Programms für einen einzelnen Mikroprozessor ist sicher nicht trivial. Die gängige Struktur besteht aus einer Hauptschleife, sie ruft die untergeordneten Module der Reihe nach auf. Dieses Konzept kann nur aufgehen, wenn die miteinander konkurrierenden Module nicht allzu zahlreich sind. Die Module haben meistens unterschiedliche oder sogar variable Periodenlängen, was enorme Schwierigkeiten mit sich bringt. Wenn eine schnelle Verarbeitungsschleife mit einer langsamen Rechenschleife im Hintergrund kooperieren muss, wie sollen beide miteinander in Einklang gebracht werden? Mit kooperativem Time-Sharing? Interrupts? Der Mikroprozessor unterstützt diverse Mechanismen, um Energie zu sparen, die natürlich nicht brach liegen sollen. Wenn dies alles und noch einiges mehr in einer klassischen Endlosschleife untergebracht werden müsste, wäre diese Endlosschleife ein halsbrecherisches Kunststück. Die Überlegungen laufen darauf hinaus, dass hier ein im Time-Sharing-Betrieb arbeitendes Echtzeit-Betriebssystem (Real-time Operating System, RTOS) geschaffen werden muss.

Ein Echtzeit-Betriebssystem bietet den Vorteil, dass der Programmcode modular aufgebaut werden kann. Die Teilkomponenten der Applikation arbeiten voneinander unabhängig an den ihnen übertragenen Aufgaben. Das RTOS ist im Hintergrund damit beschäftigt, Rechenzeiten zuzuweisen, so dass die Tasks quasi gleichzeitig abgearbeitet werden. Außerdem stellt ein Betriebssystem diverse strukturierte Methoden der gegenseitigen Kommunikation bereit, so dass die Problematik globaler Variablen umgangen wird. Verglichen mit einem konventionellen Betriebssystem ist ein RTOS überschaubar und reaktionsschnell. Alles dies sind gute Gründe, die Firmware des i-Pendels als Applikation zu schreiben, die unter einem eigenen RTOS mit dem Namen *NanoScale* läuft. Weitere Informationen zu diesem Thema sind auf der Website von GenId [1] zu finden.

Architektur

Die Architektur der Embedded-Anwendung ist, wie **Bild 1** zeigt, aus drei Subsystemen zusammengesetzt.

1. ***Kommunikation* (*Communication*)** mit der Außenwelt über eine serielle Schnittstelle. Eine Task nimmt Kommandos entgegen, eine andere Task sammelt Sendeaufträge der Applikation und schickt die zugehörigen Daten portionsweise zum UART. Diese Task übernimmt auch das Zeitmultiplexen der zu sendenden Daten. Bei den Daten handelt es sich entweder um Antworten auf empfangene Kommandos

oder um spontan gesendete telemetrische Informationen.

2. Die ***Steuerung*** (***Control***) ist eine Task mit hoher Priorität, sie wird im Abstand von 10 ms aktiv. Die Sensoren werden gelesen, der Zustand des i-Pendels wird bewertet, und daraus wird nach dem zutreffenden Algorithmus das benötigte Drehmoment berechnet. Jede Grundfunktion des i-Pendels hat einen eigenen Algorithmus: Auftrieb, statisches Gleichgewicht, Balance um den Schwerpunkt und so weiter.

3. Das ***Management*** (***Mngt***) beschäftigt sich mit allgemeinen Aufgaben wie beispielweise Aktivieren der Steuerung, Überwachen der Akkuspannung oder Blinken der LEDs.

Initialisierung und Abstraktion des Mikrocontrollers

In dem Modul *Board Support Package* (**BSP.c**) wird der Mikrocontroller initialisiert. Zuerst wird die interne Taktfrequenz definiert und die periphere I/O konfiguriert. Dann werden die Treiber für die Peripherie initialisiert: SPI-Bus und UART sowie Treiber für den Beschleunigungssensor ADXL345 und die Gyroskop-Treiber für den ISZ2510. Die beiden letztgenannten kommunizieren mit ihren gleichnamigen Sensoren über SPI. Schließlich wird das RTOS initialisiert.

Das *Board Support Package* ist gleichzeitig eine Abstraktionsschicht, die den Mikrocontroller von der übrigen Applikation abschirmt. Es enthält alle Interruptroutinen (ISRs), wobei die ISR für INT2 eine wichtige Funktion hat. Diese ISR reagiert auf das Tachometersignal der Motorsteuerung. Das Tachometer generiert einen Impuls bei jedem Achtel einer Achsenumdrehung, daraus wird die Drehzahl abgeleitet.

Die Abstraktionsschicht setzt sich aus spezifischen APIs zusammen und bietet darüberliegenden Applikation triviale Funktionen an. Dazu gehören zum Beispiel das Steuern von LEDs, das Simulieren eines EEPROM im internen Flash-Speicher, das Lesen der Akku-Spannung oder das Bedienen der Motorsteuerung.

Externe Schnittstellen

Das Modul **battery.c** liest die Akkuspannung über einen analogen Eingang des Mikrocontrollers und korrigiert den Wert mit einem Kalibrierungs-Faktor. Danach wird aus den gemessenen Werten von Strom und Spannung der Ladezustand ermittelt.

Das Modul **motor.c** macht über IC3, einen L6235, die Schnittstellen für die Motorsteuerung verfügbar. IC3 steuert den Strom, der durch die Motorwicklungen fließt. Der Strom hängt von der Spannung V_{ref} ab, die an IC3 liegt. Die Anzugskraft eines kollektorlosen Motors ist proportional zum Strom durch die Wicklungen, so dass hier zumindest theoretisch gleichzeitig das Drehmoment gesteuert wird. Dies ist eine Größe, die für das Balancieren eine zentrale Bedeutung hat.

Leider gibt es zwei Probleme, die der Lösung bedürfen. Das erste Problem besteht darin, dass der L6235 nicht im Vierquadranten-Modus betriebsfähig ist. Er steuert das Drehmoment nur, wenn die Richtung mit der Achsrotation übereinstimmt. Das zweite Problem ist das Verhalten des L6235, wenn das Drehmoment nur gering ist.

Beim Lösen des ersten Problems half uns ein intelligenter Entwicklervorschlag von STMicroelectronics. Der L6235 wird mit

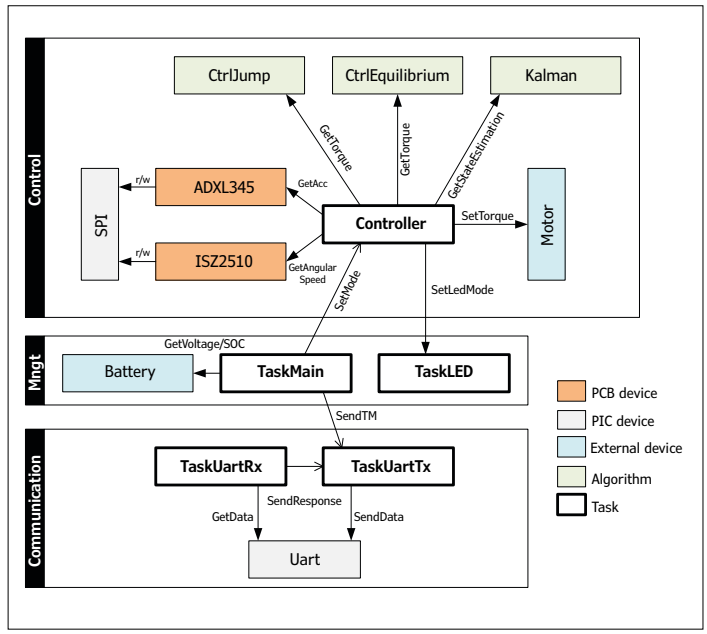


Bild 1. Übersicht über die Architektur der Firmware.

einem PWM-Signal an seinem Eingang FWD/REV gesteuert, der die Motordrehrichtung bestimmt. Der Motor ist das Glied einer H-Brücke, und der Duty-Cycle des PWM-Signals an FWD/REV bestimmt, was geschieht: Über 50 % ist gleichbedeutend mit Vorwärts, unter 50 % mit Rückwärts. Damit lassen sich in kürzesten Zeiten sowohl die Richtung als auch das Drehmoment stellen. Mehr zu dieser Methode steht in dem schon genannten Entwicklervorschlag, dem *Design Tip* von STM [2]. Nach einigem Probieren hatten wir die Kombination von Steuersignalen gefunden, die eine negative Antriebskraft bewirkt, also den Motor bremst.

Das Bearbeiten des zweiten Problems gestaltete sich schwieriger. Dafür haben wir noch keine befriedigende Lösung gefunden, so dass wir ihm vorläufig aus dem Weg gehen. Wenn das Drehmoment niedrig sein muss, ist die Steuerung des L6235 inaktiv. Ausgehend von der theoretischen Motorcharakteristik wird das oben beschriebene Verfahren angewendet, auf dieser Basis wird ein mittleres Drehmoment für die differentielle Regelzeit berechnet. Die Methode kann jedoch keine Diskontinuität ausschließen, wenn sich die Drehrichtung umkehrt. Vorläufig bedeutet dies, dass die Balance nicht geregelt wird, wenn sich die Drehzahl des Schwungrades dem Wert 0 nähert.

Sensor-Treiber

Der Treiber für den ADXL345 (IC4) steckt in dem gleichnamigen Modul **ADXL345.c**. Dies ist eine API für Sensor-Funktionen wie die Daten-Übertragungsgeschwindigkeit, den Energiesparmodus und anderes mehr. Der Treiber kommuniziert mit dem Sensor, indem er bei Bedarf Frames über SPI ausgibt. Die wichtigste Funktion ist das Lesen der letzten Beschleunigungswerte (in mg) entlang drei Achsen: `ADXL345_GetAccelerations`.

Der Treiber des ISZ2510 (IC6) arbeitet ähnlich, er befindet sich im Modul **ISZ2510.c**. Seine Hauptaufgabe ist das Lesen des letzten Werts der Winkelgeschwindigkeit (in Tausendstel °/s): `ISZ2510_GetAngularSpeed`.

```
// Compute state feedback current
torque = -K1*body_a -K2*body_w -K3*wheel_w;

// Compute torque
torque *= p->Km;
```

Bild 2. Verantwortlich für das Gleichgewicht sind zwei Zeilen des Programmcodes.

Task-Verteilung

Die Rechenleistung des Mikrocontrollers ist in mehrere Tasks aufgeteilt, die mehr oder weniger kritisch sind. Das RTOS führt dabei Regie, indem es stets die Task mit der höchsten Priorität vorzieht. `TaskLED.c` hat von allen die niedrigste Priorität, wie der Name vermuten lässt, steuert diese Task die LEDs. Zu jeder LED gehört ein Blinkgenerator, der die LED allein oder synchron mit den anderen blinken lässt.

`TaskMain.c` besitzt in der Rangfolge die höchste Priorität. Zu Beginn prüft die Task, ob die Organe und Sinne des i-Pendels bereit sind, also die Motorsteuerung arbeitet und die Sensoren antworten. Dann startet die Regelungsschleife (`Controller.c`, siehe unten), und das Durchlaufen eines Zyklus wird abgewartet. Das Signal erscheint im Abstand von 10 ms. Dann werden alle benötigten Informationen gesammelt, und die Telemetrie-Frames werden über den UART gesendet (SendTM im Schema). Die Daten bestehen aus den Sensor-Werten, dem Balance-Zustand hinter dem Kalman-Filter und den Signalen für die Motorsteuerung. Im Minutenabstand wird die Akkuspannung gelesen.

`TaskUartTx.c` schickt die Informationen des i-Pendels (*messages*) zum UART, dabei handelt es sich um Antworten auf Kommandos oder Telemetrie-Daten. Zuvor werden die Messages in einem FIFO-Puffer zwischengespeichert.

`TaskUartRx.c` verarbeitet die Kommandos, die der UART empfängt und die Informationen über den Systemzustand enthalten, sofern sie nicht zu den Telemetrie-Daten gehören. Das betrifft beispielsweise den Betriebszustand des RTOS, den Ladezustand des Akkus und so weiter.

`Controller.c` ist die Task mit höchster Priorität. Zu ihren Aufgaben gehört die Regelung der Balance, der Regelungszyklus dauert 10 ms. Ein Zyklus besteht aus folgenden Schritten:

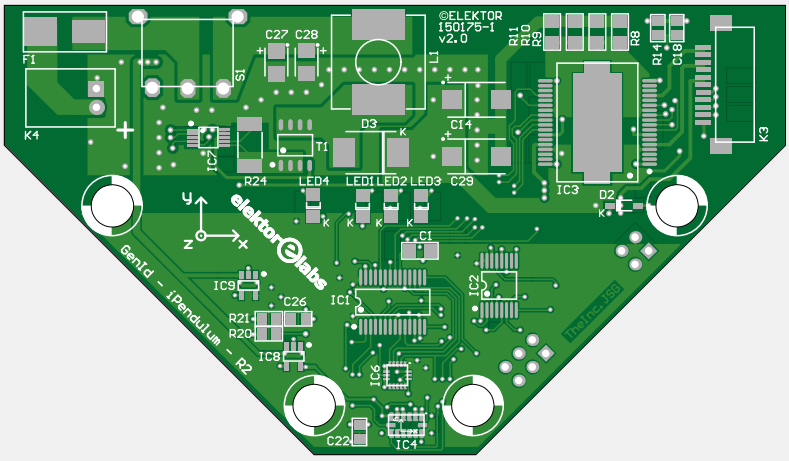
1. Lesen der Sensoren (Beschleunigungssensor und Gyroskop),
2. Schätzen des Balancezustands mit den Sensor-Werten als Eingangsgröße für das Kalman-Filter (in `KalmanFree.c`),
3. Berechnen des Setpoints für das gewünschte Drehmoment, abhängig vom Modus (`CtrlEquilibrium.c` für das Balancieren oder `CtrlJump.c` für das Aufrichten),
4. Anwenden dieses Setpoints.

Der Kern der Regelung belegt zwei Programmzeilen (**Bild 2**), die fortlaufend berechnen, mit welchem Drehmoment das i-Pendel im Gleichgewicht bleibt. Der Rest ist eigentlich nur Beiwerk. Allerdings steckt auch hier der Teufel im Detail.

Aufbauen des i-Pendels

Die Abmessungen des i-Pendels halten sich in Grenzen, dem muss auch das Platinenlayout Rechnung tragen. Die Dichte der Bauelemente, hauptsächlich SMDs, ist auf der doppelsei-

Stückliste



Widerstände:
(SMD 0805, 0,125 W, 1 %, wenn nicht anders angegeben)
R1, R21 = 10 k
R2 = 330 Ω
R3, R4, R5, R22 = 130 Ω
R6 = 100 Ω, SMD 1206, 0,25 W, 1 %
R7 = 100 k
R8, R9, R10, R11 = 1 Ω, SMD 1206, 0,25 W, 1 %
R12 = 56 k
R13 = 11 k
R14, R20 = 20 k
R15, R26 = 24k9
R16, R17, R18 = 4k7
R19 = 1 k
R23 = 215 k
R24 = 15 mΩ, SMD 2512, 1 W, 1 %
R25 = 160 k

Kondensatoren:
(SMD 0805, 50 V, 10 %, wenn nicht anders angegeben)
C1, C37, C40 = 10 µF, 16 V, SMD 1206
C2, C3, C4, C5, C6, C7, C8, C10, C21, C22, C23, C25, C26, C30, C31 = 100 n
C9, C11, C12, C17, C24, C36, C39 = 10 n
C13, C15, C33 = 220 n
C14, C29 = 68 µ, 16 V, 20 %, 0Ω05 , SMD-D
C16 = 5n6
C18 = 1n2
C19 = 33 n
C20, C32, C35, C38 = 1 µ
C27, C28 = 47 µ, 10 V, 20 %, 0Ω035 SMD-D
C34 = 100 p

Induktivität:
L1 = 5 µH, abgeschirmt, high-power

Halbleiter:
D1, D2 = 1N4148W
D3 = MBRS340T3G
LED1 = LED, SMD 1206, rot
LED2 = LED, SMD 1206, gelb
LED3, LED4 = LED, SMD 1206, grün
IC1 = dsPIC33EP128MC202-I/SS, Microchip
IC2 = SN65C3221E
IC3 = L6235PD

tigen Platine hoch.
Der Beschleunigungssensor und das Gyroskop bedürfen besonderer Maßnahmen, denn es gibt sie ausschließlich im LGA-Gehäuse (*Land Grid Array*). Ein konventioneller Lötkolben kann hier nichts ausrichten, notwendig ist ein SMD-Lötofen!
Wenn die Platine aufgebaut ist, muss der Mikrocontroller über Steckverbinder K1 mit der Firmware programmiert werden. Die Firmware steht auf der Projektseite [3] neben dem Quellcode zum freien Download bereit. Bei Bedarf kann die Firmware mit MPLAB X von Microchip modifiziert und neu compiliert werden. In der Ansicht, die **Bild 3** zeigt, ist die Platine bereits in das i-Pendel eingebaut.

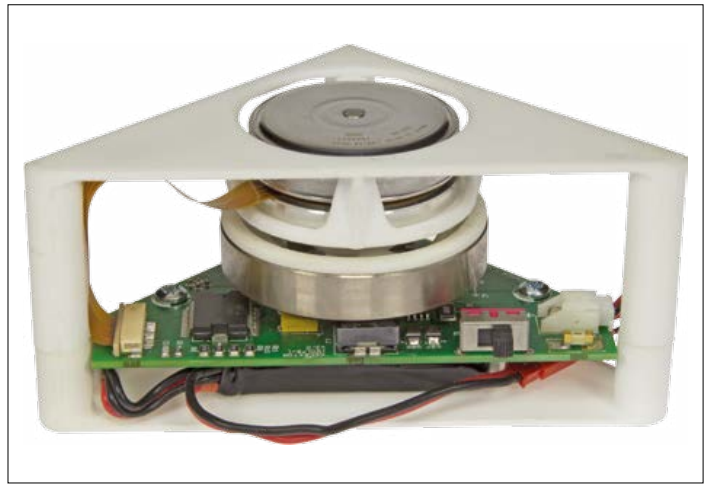


Bild 3. Das i-Pendel, von unten nach oben: Akku, Platine, Schwungrad und Motor. Der zweiteilige Rahmen wurde mit einem 3D-Drucker gefertigt.

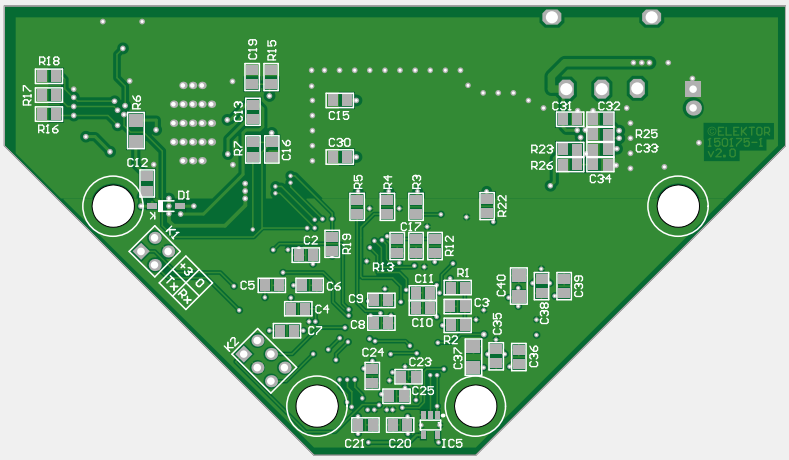
Platine testen

Vor dem Zusammenbau des i-Pendels sollten Sie die bestückte Platine testen. Als Stromversorgung können Sie ein Steckernetzteil 6...10 V oder einen geladenen LiPo-Akku an K4 anschließen. Nachdem LED4 aufleuchtet, prüft die Firmware, ob eine Kommunikation mit dem Beschleunigungssensor und dem Gyroskop möglich ist. Nun verbinden Sie den Motor mit Steckverbinder K3. Die Motorachse muss sich kurz drehen, und LED1 darf nicht leuchten. Die rote, gelbe und grüne LED auf der Platine signalisieren die Aktivitäten des i-Pendels. LED1 (rot) hat die wichtigste Bedeutung, sie zeigt an, dass ein kritischer Fehler vorliegt, so dass das i-Pendel seine Funktion verweigert. Der Blinkrhythmus gibt ersten Aufschluss über die Fehlerursache. Regelmäßiges, ununterbrochenes Blinken deutet auf niedrige Akkuspannung hin, bei zweimal kurzem Aufleuchten in Folge kann die Steuerung den Motor nicht finden, und drei oder vier Lichtimpulse mit dazwischen liegenden Pausen bedeuten, dass der Mikrocontroller vergeblich versucht, die Kommunikation mit den Sensoren aufzubauen.

Wenn das i-Pendel seinen regulären Betrieb aufgenommen hat, sind mehrere Zustände möglich. Zuerst wartet das i-Pendel auf seine stabile Lage, alle LEDs bleiben dunkel. Wenn das i-Pendel startklar ist, sich aufzurichten, beschleunigt es das Schwungrad, ein Lauflicht in Richtung der bevorstehenden Bewegung wird sichtbar. Nach dem Aufrichten blinkt LED3 so lange, wie das i-Pendel im Gleichgewicht ist.

Übersicht über die Einzelteile

Wenn die Einzelteile des i-Pendels bereit liegen, ist der Zusammenbau nicht mehr weit. Hier ist die Liste der Einzelteile:



IC4 = ADXL345BCCZ
IC5 = SN74AHC1G32DBVT
IC6 = ISZ-2510
IC7 = MAX668
IC8 = LP2985-33DBVTE4
IC9 = LP2992IM5-3.3/NOPB
T1 = FDS6680AS

Außerdem:

F1 = Sicherung 8 A, 125 V, SMD
K1 = Stiftleiste 2 · 2-polig, RM 2,54 mm
K2 = Stiftleiste 2 · 3-polig, RM 2,54 mm
K3 = Steckverbinder FFC/FPC, ZIF, 11-polig, RM 1 mm, abgewinkelt
K4 = Steckverbinder S2B-XH-A
S1 = Schiebeschalter, SPDT, abgewinkelt
Platine 150695-1

Nicht auf der Platine:

M1 = Motor, EC 45 Flat, Ø 42,9 mm, kollektorlos, 12 V, 30 W, mit Hall-Sensoren
BAT1 = Lithium-Polymer-Akku, 7,4 V, 2S 30C, 450 mAh

Mechanische Teile:

Rahmenhälfte für Platine (3D-Drucker)
Rahmenhälfte für Motor (3D-Drucker)
Nabe für Schwungrad (3D-Drucker)
Ring für Schwungrad, Ø außen = 55 mm, Ø innen = 48 mm, Höhe = 10 mm (Misumi)
Schrauben für Motor: 3 Stück M3, 6 mm lang
Schrauben für Rahmen: 6 Stück NO6X1/2

Anschlusskabel für den Akku:
JST-RCY-Stecker, männlich, mit Gehäuse
JST-XH-Stecker, weiblich, mit Gehäuse
Leitung einadrig, AWG 20, rot, 40 mm
Leitung einadrig, AWG 20, schwarz, 40 mm

Bild 4. Der Motor mit montiertem Schwungrad und der zweiteilige Kunststoffrahmen.

1. Aufgebaute und funktionsfähige Platine, hier müssen nur noch Einstellungen vorgenommen werden.
2. Lithium-Polymer-Akku 7,4 V, Abmessungen 5 · 3 · 1 cm. Wir haben einen Typ von TeamOrian verwendet, der die vielsagende Bezeichnung „7,4V Lipo 2S 30C 450mah“ trägt. Am Anschlusskabel dieses Typs ist ein JST-RCY-Stecker angebracht, im Modellbau auch als BEC-Stecker bekannt. Da dieser Stecker mit der Platine nicht kompatibel ist, muss ein Adapter zum dort befindlichen JST-XH-Steckverbinder angefertigt werden. Es gibt zwar Spezialzangen, die den Stecker auf das Kabel lötfrei aufpressen, doch sie sind nicht nur teuer, sondern auch für nur einen einzigen Steckertyp konstruiert. Deshalb erscheint es sinnvoller, die Stecker auf konventionelle Weise mit dem Lötkolben an das Kabel zu löten.
3. Drei Kunststoff-Teile: Der Rahmen besteht aus zwei Hälften, eine Hälfte nimmt die Platine und die andere den Motor auf. Das dritte Kunststoffteil ist die Nabe des Schwungrades. Die Dateien für das Anfertigen mit einem 3D-Drucker können von der Projektseite [3] heruntergeladen werden. Der Autor hat die Teile von einem 3D-Print-Service [4] anfertigen lassen. Das Material ist ein leichter, weißer Kunststoff, andere Materialien können höhere Gewichte und größere Toleranzen der Abmessungen haben.
4. Kollektorloser Motor, EC 45 Flat-Serie, Maxon-Artikelnummer 200142. Mit seiner ungewöhnlich flachen Form ist dieser Motor für das i-Pendel ideal geeignet.
5. Ein metallener Ring als Masse des Schwungrades. Der Ring hatte beim Autor den Außendurchmesser 55 mm, der Innendurchmesser betrug 48 mm und die Stärke war 10 mm. Die Artikelnummer bei Misumi [5] lautet AWSM-D-D55-V48-T10. Der komplexe Teil des Schwungrades wird mit einem 3D-Drucker aus Kunststoff angefertigt. Nur der Ring, der die Schwungmasse verkörpert, besteht aus Metall.
6. Schrauben für die Montage der Einzelteile: Drei Gewindeschrauben M3, 6 mm lang, zum Befestigen des Motors, sowie sechs selbst schneidende Kunststoffschauben für die Platinen und die Rahmenhälften. Die Abmessungen der Kunststoffschrauben richten sich nach den Gegebenheiten.

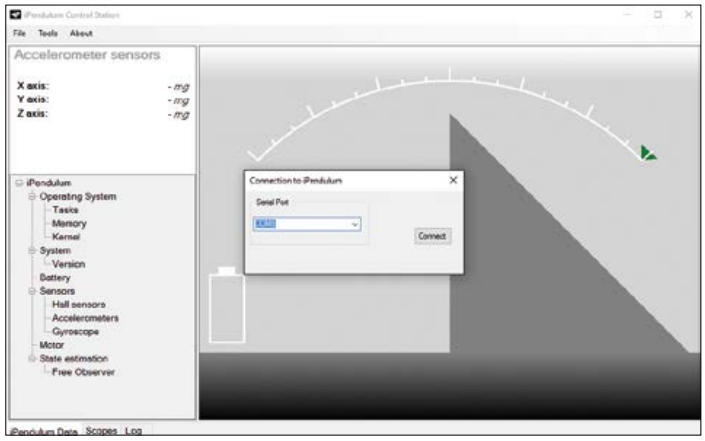


Bild 5. Das i-Pendel wird an das Diagnoseprogramm *iPendulum Control Station* gekoppelt.

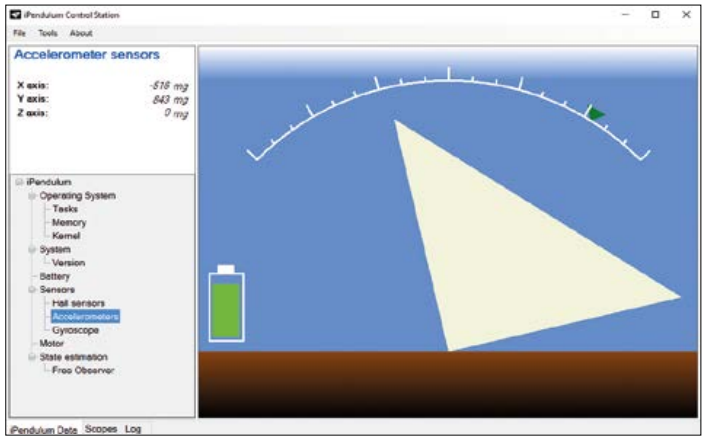


Bild 6. Im Diagnoseprogramm wird das i-Pendel stilisiert in Echtzeit dargestellt.

Mechanischer Zusammenbau

Der Zusammenbau des Schwungrades erfordert erhöhte Sorgfalt. Zuerst klemmen Sie den Metallring um die Kunststoffnabe, nehmen Sie dafür einen Schraubstock zu Hilfe. Beide Teile sind so bemessen, dass sie durch die mechanische Spannung dauerhaft verbunden bleiben. Falls sich der Ring nicht aufschieben lässt, müssen Sie die Nabe rundum leicht schmirgeln. Sollte der Ring zu weit sein, helfen einige Tropfen Sekundenkleber. Danach müssen Sie die Bohrung (4 mm Ø) für die Motorachse in das Schwungrad bohren. Hier ist Präzisionsarbeit zu leisten, denn die Bohrung muss sich exakt im Mittelpunkt befinden, und sie muss exakt senkrecht zur Ebene der Nabe ausgerichtet sein! Wenn möglich, benutzen Sie eine solide, auf festem Sockel stehende Standbohrmaschine. Wir empfehlen, einige Probebohrungen in weniger wertvolle Werkstücke vorzunehmen. Die geringste Abweichung oder Asymmetrie hat später ein Ungleichgewicht und Wackelerscheinungen zur Folge, was beim i-Pendel unbedingt vermieden werden muss.

Befestigen Sie zuerst den Motor mit drei Schrauben an einer Rahmenhälfte, das Kabel führen Sie durch die Aussparung in der Motoraufhängung (siehe **Bild 3**). Setzen Sie das Schwungrad auf die Achse (**Bild 4**), und achten Sie darauf, dass Sie auf die Mitte des Motorgehäuses drücken. Das Belasten des Außenrands kann zur Beschädigung des Motors führen. Wenn beides nicht exakt zusammenpasst, muss entweder die Bohrung geweitet werden, oder die Teile werden mit einem Tropfen Sekundenkleber fixiert.

Der Abstand zwischen dem Schwungrad auf dem Motor und der Induktivität L1 auf der Platine ist äußerst knapp bemessen. Dieses Problem haben wir mit einem Zwischenring zwischen dem Motor und der Motoraufhängung gelöst. Der Abstand zwischen dem Schwungrad und Induktivität L1 wird so um etwa 1 mm vergrößert. Inzwischen haben wir die 3D-Drucker-Dateien angepasst, so dass der Zwischenring entfallen kann.
Damit sind die schwierigsten Schritte des mechanischen Zusammenbaus erledigt. Befestigen Sie jetzt die Platine mit vier selbst schneidenden Schrauben an der anderen Rahmenhälfte. Verbinden Sie das Motorkabel mit Steckverbinder K3 und montieren Sie die Rahmenhälften mit den übrigen Schrauben gegeneinander.

Herzlichen Glückwunsch! Ihr i-Pendel ist nun fast startklar. Schieben Sie nur noch den geladenen LiPo-Akku an seinen Platz und schließen Sie ihn an.

Jetzt geht es los!
Wenn Sie das i-Pendel mit einer Seitenfläche (ganzes Dreieck) flach auf die Unterlage legen, kann es sich nicht aufrichten. Es wechselt dann automatisch in den Sleep-Modus. Soll sich das i-Pendel selbst aufrichten, muss die Seite, auf der es balanciert, mit der Unterlage in Kontakt sein. Oder einfacher ausgedrückt: Legen Sie das i-Pendel mit einer kurzen Dreieckseite auf den Tisch und schalten Sie die Betriebsspannung ein. Das i-Pendel bleibt fünf Sekunden in der Ruhelage, dann versucht es, sich aufzurichten: Das Schwungrad läuft an, steigert seine Drehgeschwindigkeit und bremst dann abrupt. Dadurch wirkt ein mechanischer Impuls auf das i-Pendel ein, der es auf seinen Gleichgewichtspunkt stellt. Anschließend hält es sich selbst im Gleichgewicht. Sie können das i-Pendel auch von Hand auf den Gleichgewichtspunkt stellen. Solange der Winkel zwischen der Unterlage und der kurzen Dreieckseite ausreichend groß ist, entwickelt der Motor ein genügend hohes Drehmoment, um das Gleichgewicht zu halten. Wenn Sie das i-Pendel aus dem Gleichgewicht bringen, wird es versuchen, das Gleichgewicht wieder herzustellen. Gelingt ihm dies nicht, fällt es auf eine der kurzen Seiten. Nach fünf Sekunden folgt ein neuer Versuch, sich selbst wieder aufzurichten.

Diagnose-Programm
Das Programm *iPendulum Control Station*, das wir ebenfalls auf die Projektseite [3] eingestellt haben, wurde in C# für Windows geschrieben. Über eine serielle Datenverbindung können beim i-Pendel diverse Zusammenhänge und Fakten erkundet werden. Vorwiegend dient das Programm der Diagnostik, der Autor hat es jedoch auch während der Entwicklung eingesetzt, um das Verhalten des i-Pendels in Echtzeit zu analysieren.

Starten Sie das Programm und gehen Sie zum Menü *Tools → Connect*. Geben Sie, wie **Bild 5** zeigt, den COM-Port ein, an dem Sie das i-Pendel angeschlossen haben. Wenn die Verbindung hergestellt ist, erscheint die Meldung *Connect to Pendulum*. Im Hauptfenster, erkennbar an der Farbe Blau, ist das stilisierte i-Pendel und der Winkel zu sehen, den es zur Unterlage einnimmt (**Bild 6**). Links unten im Fenster sind drei Tabs vorhanden. Nach dem Programmstart ist immer der Tab *iPendulum Data* aktiv, in dem die i-Pendel-Informationen in Echtzeit betrachtet werden können.

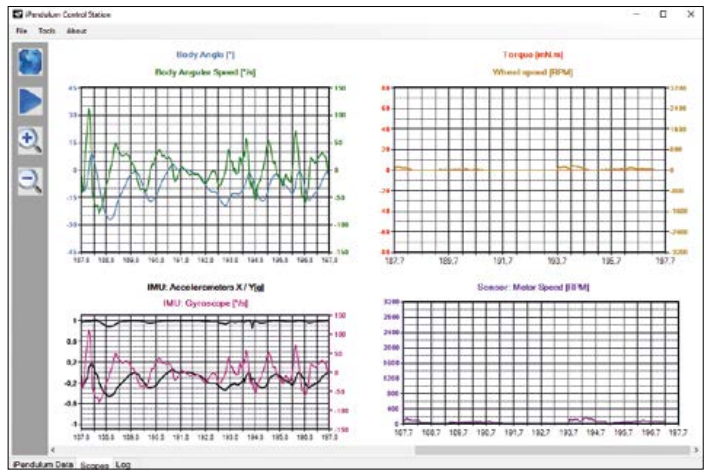


Bild 7. Die Grafiken stellen die Abläufe und ihre Messdaten ebenfalls in Echtzeit dar.

Programm-Aktionen
Wenn Sie das i-Pendel von Hand bewegen, können Sie die Reaktionen auf dem Bildschirm beobachten. Im linken Teil des Fensters sehen Sie einen Baum, über den Sie die zu betrachtenden Informationen wählen.

- *Operating System*: Informationen zum RTOS (Version, Speicher, Tasks, CPU-Last),
- *System*: Allgemeine Informationen zur Firmware,
- *Battery*: Akku-Spannung und Ladezustand in %,
- *Sensors*: Daten, die unmittelbar von den Sensoren kommen,
- *Motor*: Drehmoment, das der Motor aktuell entwickelt,
- *State*: Balance-Status, abgeschätzt mit dem Kalman-Filter.

Der Tab ***Scopes*** stellt die Telemetrie-Daten dar, die das i-Pendel übertragen hat. Dort sind vier miteinander synchronisierte Grafiken zu sehen, die Zeitachse hat in jeder Grafik einen übereinstimmenden Maßstab (**Bild 7**).

1. Links oben der geschätzte Balance-Status,
2. rechts oben das Drehmoment und die Drehzahl des Motors,
3. links unten die Daten des Beschleunigungssensors und des Gyroskops,
4. rechts unten die Geschwindigkeit gemessen vom Motorcontroller, unabhängig von der Motordrehrichtung.

Die Grafiken werden dynamisch aktualisiert, sobald Telemetrie-Daten empfangen werden. Mit dem Mausrad oder den Schaltflächen am linken Fensterrand lassen sich die Grafiken zoomen. Auch das Scrollen mit dem horizontalen Scrollbalken oder durch Verschieben mit der Maus ist möglich.
Im Tab ***Log*** können die Daten geloggt werden. Sämtliche Daten, die das Programm vom i-Pendel empfängt, werden registriert. Geben Sie einen Namen für die Logging-Datei ein und klicken Sie auf *Start Logging*.

Akkuspannung kalibrieren
Der Ladezustand des Akkus ist eine für den Betrieb des i-Pendels wichtige Größe. Wenn die Akkuspannung unter einen Mindestwert sinkt, kann der Akku beschädigt werden. Der Grenz-

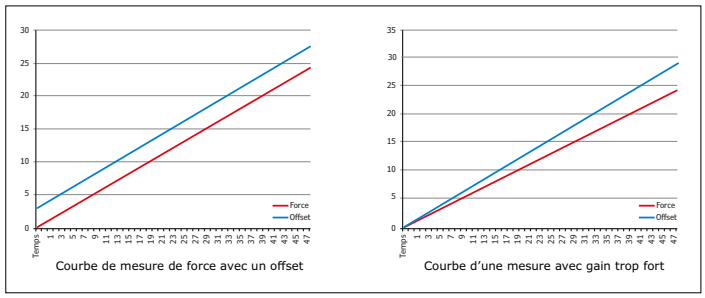


Bild 8. Die kalibrierten Geraden erscheinen in Rot, die nicht kalibrierten Geraden in Blau.

wert liegt bei Lithium-Polymer-Akkus bei 3,7 V pro Zelle, bei einem Akku mit zwei Zellen folglich bei 7,4 V. Im Menü *Tools → Calibration → Battery* kann die vom i-Pendel gemessene Spannung korrigiert werden. Dort sind der nicht korrigierte und der korrigierte Wert zu sehen. Die mit einem Multimeter am Akku gemessene Spannung wird in das Feld des korrigierten Werts eingetragen.

Sensoren kalibrieren

Das Gyroskop und der Beschleunigungssensor dienen dem i-Pendel dazu, fortlaufend die Drehgeschwindigkeit und den Raumwinkel zur Unterlage abzuschätzen. Die Sensoren sind ebenso wie andere Messsysteme mit Fehlern behaftet. Beim Einsatz im i-Pendel müssen nur der Offset und die Skalierung korrigiert werden (**Bild 8**), die sonstigen Fehler sind hier unkritisch.

Der Offset ist eine Systemkonstante, die sich auf alle Messungen gleichartig auswirkt. Um den Offset zu eliminieren, muss der Sensor so positioniert werden, dass das Ausgangssignal gleich 0 sein müsste. Der dann am Sensor-Ausgang gemessene Wert ist der Offset.

Der Offset des Gyroskops lässt sich unkompliziert ermitteln: Das i-Pendel wird in seine Ruheposition gebracht, der Offset wird abgelesen. Die Korrektur ist wichtig, denn dieser Offset hat weitreichende Folgen. Das Gyroskop misst die Rotationsgeschwindigkeit, es soll jedoch die Position bestimmt werden. Die Position ist das Integral der Rotationsgeschwindigkeit. Das Integral einer Konstante, also des Offsets, ergibt einen Positionsfehler, der bis Unendlich wächst. Hinzu kommt noch das Driften des Offsets mit der Temperatur und mit der Zeit.

Beim Beschleunigungssensor sind die Zusammenhänge etwas komplizierter. Hier muss der Offset eigentlich bestimmt werden, indem der Sensor exakt senkrecht zur Horizontalen ausgerichtet wird, doch das ist nicht ganz einfach. Die Lösung dieses Problems besteht darin, den Mittelwert aus Beschleunigungswerten zu bilden, die zu zwei asymmetrischen Positionen gehören. In den Positionen müssen sich die Abweichungen bezogen auf die Horizontale gegenseitig kompensieren. Der Offset des Beschleunigungsmessers hat auf das Gyroskop nur wenig Einfluss. Das Signal ist unmittelbar ein Maß für die Stellung des i-Pendels, eine mathematische Operation wie das Integrieren ist hier nicht nötig.

Der Skalierungsfehler wirkt sich auf die Verstärkung des Systems aus. Der gemessene Wert hat eine Verstärkung ungleich 1, bezogen auf den tatsächlichen Wert. Beim i-Pendel wird dieser Fehler nicht korrigiert. Um den Verstärkungsfehler zu messen, müsste sich das i-Pendel mit einer exakt bekannten Geschwindigkeit drehen. Es fehlt jedoch ein Messsystem, das dies leisten könnte. Beim Beschleunigungsmesser liegen die Randbedingungen anders. Hier ist der benötigte Referenzwert, nämlich die Schwerkraft, immer bekannt. Abhängig von der Lage des i-Pendels, entweder flach auf der Unterlage oder auf einer kurzen Seite, gibt der Sensor entweder die Beschleunigung 0 oder die Gravitationskonstante g an.

Die Zusammenhänge sind komplex, doch in der Firmware ist eine Kalibrier-Prozedur implementiert, die Sie durch die Klippen lotst. Gehen Sie zum Menü *Tools → Calibration → IMU* und bringen Sie das i-Pendel in eine genau definierte Position, wenn Sie dazu aufgefordert werden. Folgen Sie einfach den Anweisungen auf dem Bildschirm.

Zum Schluss

Damit sind wir am Ende dieses Beitrags angekommen. Wir hoffen, dass wir Ihre Entdeckerfreude geweckt haben und einen Zipfel des Schleiers lüften konnten, der die moderne Regelungstechnik umhüllt. Welchen Nutzen hat dieses Projekt? Das hängt von dem Aspekt ab, unter dem Sie es betrachten. Wenn es Ihnen Freude bereitet, mit innovativen Technologien in Tuchfühlung zu bleiben, hat das Projekt sein Ziel erreicht. Die Regelungstechnik ist ein Teilgebiet der Automatisierung, sie begleitet unser Leben auf Schritt und Tritt. Die grundlegenden Gesetzmäßigkeiten wendet auch die Natur an, sie ist in mancherlei Hinsicht ein Vorbild. Regelungsmechanismen wie im i-Pendel sind heute integraler Bestandteil von Fahrzeugen, Kommunikationssystemen, Fluggeräten, Raketen und vielem mehr. Ebenso sind sie in der Welt der biologischen Lebewesen verankert, beispielsweise bei den Interaktionen der Eiweiße in den Zellen, beim Aufrechterhalten der Körpertemperatur und beim Regeln des Blutdrucks.

Noch ein letzter Hinweis: Sämtliche Software, die wir hier beschrieben haben, ist Open-Source. Sie können sie nicht kommerziell nutzen, analysieren, modifizieren und ausbauen. ⏮

(150695)gd

Weblinks

[1] www.genid.eu/dp/?q=Products\NanoScale\WhatIsNanoScale

[2] www.st.com/st-web-ui/static/active/en/resource/technical/document/design_tip/DM00052458.pdf

[3] Platine, Firmware, 3D-Drucker-Dateien (Rahmen und Nabe des Schwungrades) und Diagnose-Programm: www.elektormagazine.de/150695

[4] 3D-Print-Service, den der Autor nutzte: www.sculpteo.com

[5] Misumi: www.misumi-europe.com